Trabajo Práctico Obligatorio Integrador

Paradigma Orientado a Objetos

Clínica Veterinaria - Informe Final

Integrantes:

1. Melina Bovarines - N° de legajo: 1226714
2. Giovanna Manograsso - N° de legajo: 1230393
3. Manuela Torres - N° de legajo: 1220169

Profesor: Maria Angela Leon

Fecha de entrega: 07/11/26

Cuatrimestre: 2° Año: 2026

UNIVERSIDAD ARGENTINA DE LA EMPRESA

FACULTAD DE INGENIERÍA Y CIENCIAS EXACTAS

Modificaciones realizadas sobre el diagrama de clases original

A partir del primer diagrama de clases, fuimos realizando diferentes modificaciones para ampliar las funcionalidades del sistema de gestión de la clínica veterinaria y organizar mejor las responsabilidades de cada clase.

En el módulo de turnos incorporamos la clase ServicioTurnos para centralizar las operaciones de asignación, reprogramación y cancelación de turnos, así como también el registro de las atenciones. Además, agregamos métodos en las clases Veterinario y Turno para mejorar el control de disponibilidad y la gestión de los horarios de atención.

Para el módulo de consultas, agregamos más detalles sobre las operaciones relacionadas con el registro de tratamientos, las vacunas aplicadas y los insumos utilizados durante cada atención veterinaria.

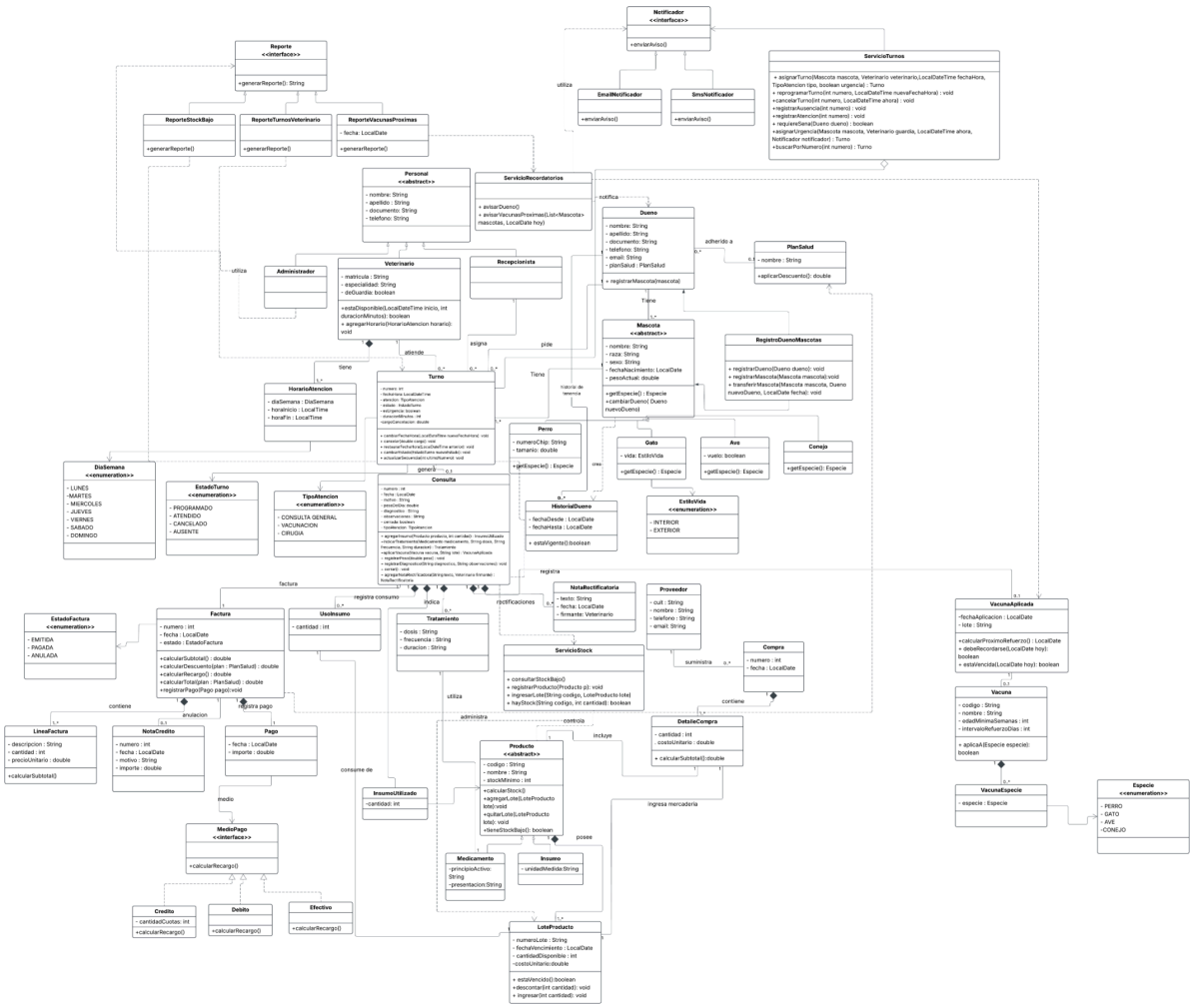
La parte de facturación también tuvo modificaciones. Ampliamos las responsabilidades de la clase Factura y detallamos las operaciones de los distintos medios de pago, para organizar mejor los cálculos y el registro de los pagos.

En el módulo de stock incorporamos nuevas operaciones en ServicioStock, Producto y LoteProducto para mejorar la administración del inventario. Esto incluye el ingreso de productos, la gestión de los lotes y el control de sus vencimientos.

Respecto al módulo de vacunas, agregamos funcionalidades para controlar los vencimientos y generar recordatorios de los refuerzos. Además, incorporamos la clase RegistroDuenoMascotas, que permite centralizar el registro de los dueños, las mascotas y las transferencias de titularidad.

Por último, ajustamos algunos atributos, métodos y relaciones entre las clases para representar de una manera más completa el funcionamiento del sistema.

Con estos cambios, el diagrama muestra con mayor detalle cómo está organizado el sistema, qué responsabilidades tiene cada clase y qué operaciones se pueden realizar en los distintos módulos.



link: https://lucid.app/lucidchart/81295d27-a414-4167-be68-2fdf4566be32/view

Modificaciones realizadas en el diagrama de casos de uso

A partir del análisis del diagrama de casos de uso original, realizamos diferentes modificaciones para ampliar las funcionalidades del Sistema de Patitas y completar los procesos relacionados con la gestión veterinaria.

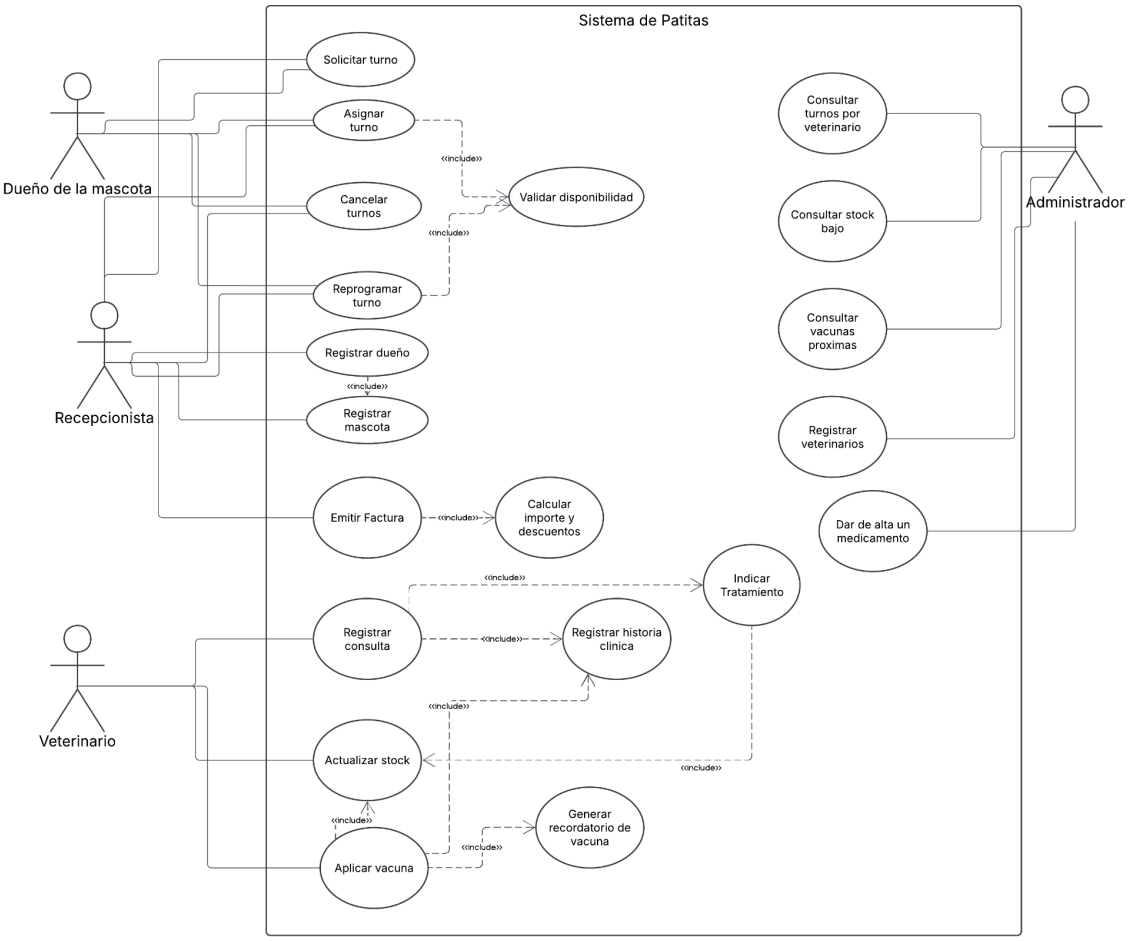
Dentro de los cambios realizados, sumamos nuevos casos de uso, como Registrar veterinarios, Dar de alta un medicamento e Indicar tratamiento. De esta forma, se ampliaron las operaciones disponibles para el administrador y el veterinario, contemplando la gestión de profesionales, medicamentos y tratamientos.

En las relaciones entre los casos de uso también hicimos algunos ajustes. Agregamos relaciones de tipo «include», como la que vincula Registrar dueño con Registrar mascota, ya que ambos procesos están relacionados al momento de registrar una mascota. A su vez, incorporamos la indicación del tratamiento dentro del proceso de registrar una consulta.

El proceso de vacunación fue otro de los aspectos que modificamos. Agregamos la actualización del stock y el registro de la vacuna en la historia clínica, de manera que ambas informaciones se mantengan actualizadas después de realizar una vacunación.

Por último, cambiamos la distribución de algunos casos de uso y sus relaciones para facilitar la lectura del diagrama, sin modificar los actores principales ni el límite del sistema.

De esta manera, quedaron contempladas más operaciones dentro del diagrama, incluyendo algunas que no estaban representadas inicialmente, y se pueden observar mejor las distintas interacciones de los usuarios con el sistema.



link: https://lucid.app/lucidchart/6c2bc8f8-9593-4db8-a5ea-0cd584aa0dcb/view

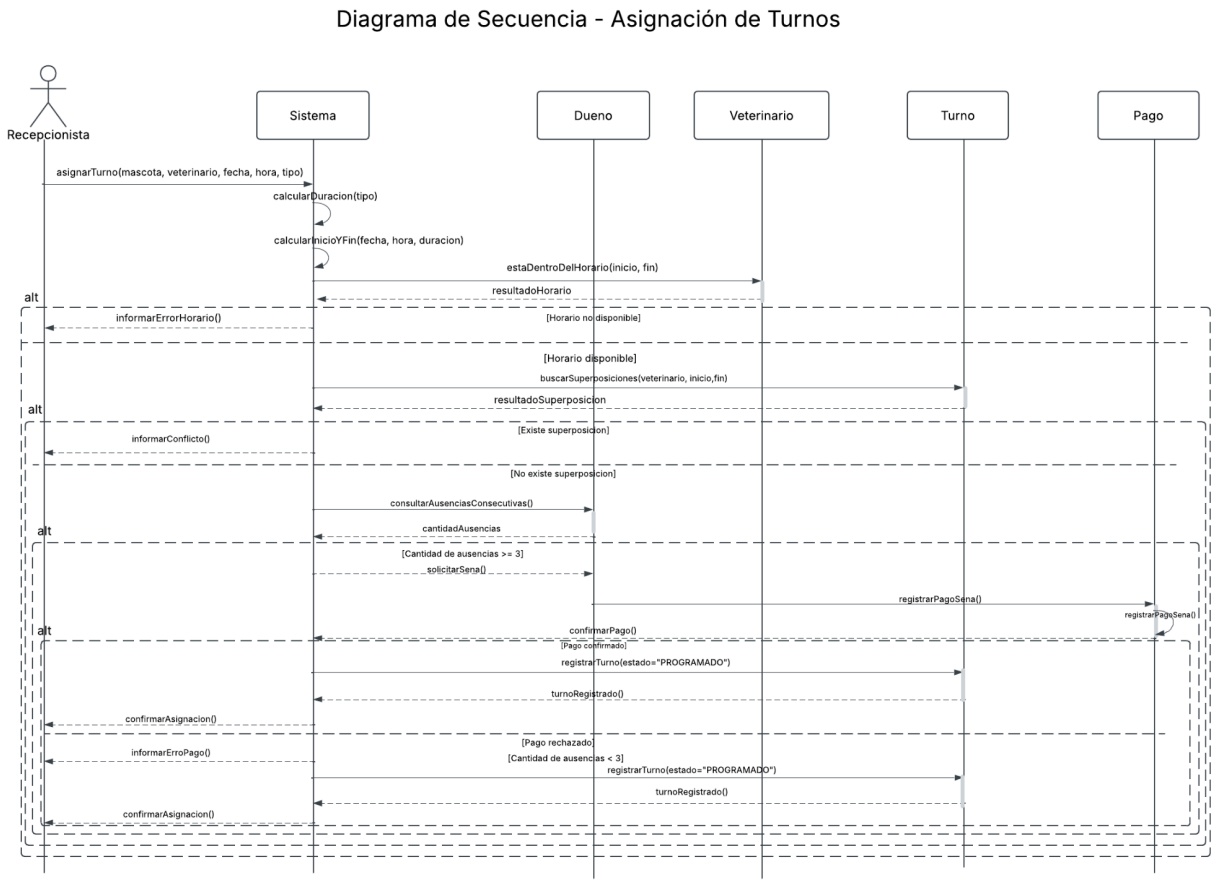
Modificaciones del diagrama de secuencia asignar turno

En el diagrama de secuencia hicimos algunos cambios para mostrar con más detalle cómo se lleva a cabo la asignación de turnos veterinarios. Uno de ellos fue agregar el cálculo del horario de inicio y finalización del turno a partir de su duración. Una vez obtenido ese horario, el sistema verifica que se encuentre dentro del horario de atención del veterinario. También se agregó la búsqueda de posibles superposiciones con otros turnos del mismo veterinario, para evitar que se asignen dos turnos en el mismo horario.

Otro aspecto que incorporamos fue el control de las ausencias consecutivas del dueño. Si la cantidad de ausencias es igual o superior a tres, el sistema solicita el pago de una seña antes de continuar con la asignación. Para este caso, contemplamos tanto la confirmación como el rechazo del pago. Si no se concreta, el sistema informa el error y no continúa con la asignación. En cambio, si el pago es

confirmado, o si el dueño tiene menos de tres ausencias consecutivas, se registra el turno con estado "PROGRAMADO" y se confirma la asignación al recepcionista.

Con esta actualización, quedan representadas las distintas situaciones que pueden surgir durante el proceso de asignación y las respuestas del sistema ante cada una de ellas.



link: https://lucid.app/lucidchart/92f49756-9819-4f3a-9d21-9f32191880e2/view

Plan de pruebas funcionales

Plan de pruebas funcionales

Dificultades encontradas

Durante el desarrollo del proyecto nos encontramos con distintas dificultades, tanto a nivel técnico como organizativo, que tuvimos que resolver para lograr el funcionamiento integrado del sistema.

1. Organización e integración del trabajo en equipo

Una de las principales dificultades fue trabajar de manera simultánea en distintas ramas de GitHub. En algunos momentos surgieron inconvenientes al realizar los merge, traer los cambios de otros integrantes o integrar clases que dependían de funcionalidades desarrolladas por otros compañeros. Esto requirió revisar los cambios, resolver conflictos y coordinar la integración para evitar perder código o generar inconsistencias.

2. Relación entre las clases y distribución de responsabilidades
Al implementar las funcionalidades, surgieron dudas sobre qué clase debía encargarse de determinadas operaciones. Por ejemplo, la asignación de turnos, el control del stock y la facturación involucraban varias clases relacionadas entre sí. Tuvimos que revisar el diseño original y reorganizar algunas responsabilidades, incorporando clases de servicio para evitar concentrar demasiada lógica en una sola clase.

3. Implementación de las reglas de negocio
Otra dificultad importante fue trasladar las reglas de negocio de la consigna al código Java. Algunas funcionalidades requerían contemplar distintas condiciones, como el control de tres ausencias consecutivas, el cobro de señas, las cancelaciones tardías, las urgencias y la validación de disponibilidad de los veterinarios. Fue necesario analizar cada situación y contemplar tanto los casos correctos como los posibles errores.

4. Manejo de fechas, horarios y validaciones
Tuvimos dificultades al trabajar con fechas y horarios, especialmente al calcular la duración de los turnos, verificar que se encuentren dentro del horario de atención y evitar superposiciones. También fue necesario contemplar situaciones particulares, como los días en los que no hay atención y las urgencias que requieren reorganizar otros turnos.

5. Persistencia de datos mediante archivos
La implementación de la persistencia también presentó dificultades, ya que debíamos lograr que la información continuara disponible después de cerrar el programa. Además, tuvimos que contemplar situaciones en las que los archivos estuvieran dañados o no pudieran leerse correctamente, evitando que estos inconvenientes impidieran iniciar el sistema.

6. Pruebas e integración final
Por último, otra dificultad que tuvimos fue comprobar que todas las funcionalidades funcionaran correctamente en conjunto. Algunas operaciones dependían de otras, por lo que no era suficiente con verificar cada método por separado. Por ejemplo, al confirmar una consulta teníamos que controlar el stock, cerrar la atención y generar la factura. Esto nos llevó a realizar pruebas unitarias y funcionales, detectar errores y hacer las correcciones necesarias hasta lograr un funcionamiento más consistente.
A partir de esta experiencia, pudimos ver la importancia de planificar el desarrollo, mantener una buena comunicación entre los integrantes del grupo y realizar pruebas durante todo el proceso, sin dejar todas las verificaciones para el final.

Distribución y organización de las tareas
Para llevar adelante el desarrollo del sistema, organizamos el trabajo utilizando Git y GitHub, dividiendo las funcionalidades en distintas ramas (*branches*) para facilitar el trabajo en equipo y permitir que cada integrante pudiera desarrollar una parte específica del proyecto de manera independiente.
En primer lugar, creamos la rama principal main, donde se integraron las versiones finales y revisadas del sistema. Además, creamos una rama feature-documentacion, destinada a desarrollar y almacenar la documentación inicial del proyecto, incluyendo los diagramas UML y las reglas de negocio.
Posteriormente, dividimos el desarrollo en tres ramas principales, cada una encargada de un conjunto de funcionalidades:

1. Rama feature/duenos-mascotas
Esta rama estuvo destinada al desarrollo de las clases y funcionalidades relacionadas con los dueños y sus mascotas. Entre las principales tareas realizadas se encuentran:

- Creación de las clases Dueno, Mascota, Perro, Gato, Ave e HistorialDueno.
- Implementación del registro y consulta de dueños y mascotas.
- Desarrollo de la funcionalidad de cambio de dueño, conservando el historial clínico de la mascota.
- Desarrollo de las clases relacionadas con las consultas clínicas, los tratamientos y las vacunas aplicadas.
- Implementación de validaciones correspondientes a la especie, la edad mínima y el peso de las mascotas.

2. Rama feature/turnos-consultas
Esta rama se enfocó en la gestión del personal veterinario, los turnos y las consultas médicas. Las tareas principales fueron:

- Creación de las clases Personal, Veterinario y Turno, junto con la definición de los horarios de atención.
- Implementación de las funcionalidades para asignar, reprogramar y cancelar turnos.
- Validación de la duración de los turnos, los horarios de atención y la existencia de superposiciones.
- Control de ausencias consecutivas de los clientes y de las señas correspondientes.
- Registro de las atenciones veterinarias y de los tratamientos realizados.

3. Rama feature/stock-facturacion-reportes
Esta rama estuvo orientada a la administración del stock, la facturación y la generación de reportes del sistema. Se desarrollaron las siguientes funcionalidades:

- Creación de las clases Producto, LoteProducto, UsoInsumo y ServicioStock.
- Implementación del sistema de facturación, incluyendo facturas, líneas de factura y medios de pago.
- Desarrollo de reportes relacionados con los turnos, el stock bajo y las vacunas próximas a vencer o aplicarse.
- Implementación de recordatorios y de las interfaces Reporte y Notificador.
- Desarrollo de la persistencia de datos mediante archivos.
- Integración de las funcionalidades necesarias para el funcionamiento general del sistema.

Una vez finalizado el trabajo de cada parte, avanzamos con la integración de las distintas ramas para reunir las funcionalidades implementadas. Durante esta etapa, desarrollamos el menú principal dentro de la carpeta vista, con el propósito de facilitar la interacción con el sistema y permitir probar las diferentes opciones disponibles.

Luego, nos enfocamos en el controlador, cuya función es conectar la interfaz con las clases del modelo, además de gestionar las operaciones y la persistencia de los datos.

Finalmente, realizamos las pruebas unitarias de las distintas funcionalidades y elaboramos el plan de pruebas para evaluar el comportamiento del sistema. A medida que avanzamos, identificamos errores

e inconvenientes a partir de los resultados obtenidos y realizamos las correcciones necesarias para asegurar que los distintos módulos funcionaran correctamente en conjunto.

Todo este proceso permitió reunir el trabajo realizado por cada integrante en un único sistema, combinando las tareas desarrolladas en paralelo y verificando que las funcionalidades pudieran trabajar de manera integrada.

Reflexión del grupo

La realización de este proyecto fue una experiencia que nos permitió poner en práctica los conocimientos adquiridos durante la materia de Programación Orientada a Objetos. A lo largo del trabajo nos enfrentamos a distintos desafíos relacionados con el diseño de clases, la implementación de reglas de negocio, el manejo de archivos, el uso de Git y la conexión de las diferentes partes del sistema.

Si bien el proyecto presentó varias dificultades y en algunos momentos nos resultó complejo, nos ayudó a comprender que desarrollar un sistema no consiste solamente en escribir código. También implica analizar un problema, organizar las tareas, definir responsabilidades y conseguir que todos los componentes funcionen correctamente en conjunto.

Uno de los aprendizajes más importantes fue el trabajo colaborativo mediante Git y GitHub. Tuvimos que organizarnos, distribuir las funcionalidades y aprender a integrar los cambios realizados por cada integrante. Además, pudimos reconocer la importancia de realizar pruebas durante el desarrollo, tanto para detectar errores como para comprobar que el sistema cumpliera con las reglas de negocio establecidas.

Por otra parte, el proyecto nos permitió identificar aquellos temas que todavía necesitamos reforzar, especialmente en lo relacionado con la organización del código, la lógica de programación y la resolución de errores. A pesar de los inconvenientes que surgieron, logramos desarrollar un sistema que reúne las principales funcionalidades solicitadas y que nos permitió acercarnos a una experiencia más cercana al desarrollo de software en un contexto real.

En definitiva, consideramos que fue una experiencia valiosa para nuestra formación, ya que pudimos reconocer lo que aprendimos, identificar los aspectos que todavía debemos mejorar y comprender mejor todo lo que implica llevar adelante un proyecto de software en equipo.